

Name: Moksh J Lalwani

Roll No.: 23BCP096

Division: 2 (G3)

Course: BlockChain Technology

Course Code: 20CP405P

Semester: 7th

Assignment - 1

Lab-1: implement doubly linked list in append only mode. Now try to generate hash of each block and connect them using the hash values.

Code:

```
#include <iostream>
#include <string>
#include <functional>

using namespace std;

class Block {
public:
    int data;
    string previousHash;
    string currentHash;
    Block* next;

    Block(int data, string previousHash) {
        this->data = data;
        this->previousHash = previousHash;
        this->currentHash = calculateHash();
        this->next = NULL;
    }

    string calculateHash() {
        hash<string> hasher;
        string input = to_string(data) + previousHash;
        return to_string(hasher(input));
    }
};
```

```

void insertTail(Block* &head, Block* &tail, int data) {
    string prevHash = "0";

    if (tail != NULL)
        prevHash = tail->currentHash;

    Block* newBlock = new Block(data, prevHash);

    if (head == NULL) {
        head = tail = newBlock;
    } else {
        tail->next = newBlock;
        tail = newBlock;
    }
}

void printBlockchain(Block* head) {

    if (head == NULL) {
        cout << "Blockchain is Empty." << endl;
        return;
    }

    Block* temp = head;
    int blockNo = 0;

    while (temp != NULL) {

        cout << "-----" << endl;
        cout << "Block Number : " << blockNo++ << endl;
        cout << "Data          : " << temp->data << endl;
        cout << "Previous Hash: " << temp->previousHash << endl;
        cout << "Current Hash : " << temp->currentHash << endl;

        temp = temp->next;
    }

    cout << "-----" << endl;
}

bool verifyBlockchain(Block* head) {

    if (head == NULL)
        return true;

    Block* temp = head->next;
    Block* prev = head;

    while (temp != NULL) {

        if (temp->previousHash != prev->currentHash)

```

```

        return false;

        prev = temp;
        temp = temp->next;
    }

    return true;
}

int getLength(Block* head) {

    int count = 0;

    while (head != NULL) {
        count++;
        head = head->next;
    }

    return count;
}

int main() {

    Block* head = NULL;
    Block* tail = NULL;

    insertTail(head, tail, 10);
    insertTail(head, tail, 20);
    insertTail(head, tail, 60);

    while (true) {

        cout << "\n Blockchain Menu " << endl;
        cout << "1. Add Block" << endl;
        cout << "2. Display Blockchain" << endl;
        cout << "3. Verify Blockchain" << endl;
        cout << "4. Blockchain Length" << endl;
        cout << "5. Exit" << endl;

        int choice;
        cout << "Enter Choice: ";
        cin >> choice;

        switch (choice) {

            case 1: {
                int data;
                cout << "Enter Data: ";
                cin >> data;

                insertTail(head, tail, data);
            }
        }
    }
}

```

```

        cout << "Block Added Successfully.\n";
        break;
    }

    case 2:
        printBlockchain(head);
        break;

    case 3:

        if (verifyBlockchain(head))
            cout << "Blockchain is VALID." << endl;
        else
            cout << "Blockchain has been TAMPERED." << endl;

        break;

    case 4:
        cout << "Total Blocks: " << getLength(head) << endl;
        break;

    case 5:
        cout << "Exiting..." << endl;
        return 0;

    default:
        cout << "Invalid Choice." << endl;
    }
}

return 0;
}

```

Output:

```
Blockchain Menu
1. Add Block
2. Display Blockchain
3. Verify Blockchain
4. Blockchain Length
5. Exit
Enter Choice: 1
Enter Data: 110
Block Added Successfully.

Blockchain Menu
1. Add Block
2. Display Blockchain
3. Verify Blockchain
4. Blockchain Length
5. Exit
Enter Choice: 2
-----
Block Number : 0
Data : 10
Previous Hash: 0
Current Hash : 9365741972668532540
-----
Block Number : 1
Data : 20
Previous Hash: 9365741972668532540
Current Hash : 7569743944453174871
-----
Block Number : 2
Data : 60
Previous Hash: 7569743944453174871
Current Hash : 14274113220143506793
-----
Block Number : 3
Data : 110
Previous Hash: 14274113220143506793
Current Hash : 2767799826712099968
-----
```

```
Blockchain Menu
1. Add Block
2. Display Blockchain
3. Verify Blockchain
4. Blockchain Length
5. Exit
Enter Choice: 3
Blockchain is VALID.

Blockchain Menu
1. Add Block
2. Display Blockchain
3. Verify Blockchain
4. Blockchain Length
5. Exit
Enter Choice: 4
Total Blocks: 4

Blockchain Menu
1. Add Block
2. Display Blockchain
3. Verify Blockchain
4. Blockchain Length
5. Exit
Enter Choice: 5
Exiting...
(base) krishasheta@Devs-MacBook-Air LAB %
```

Observation:

In this experiment, I implemented a blockchain structure using a linked list in append-only mode. Each block stores its data, current hash, and the hash of the previous block. We observed that every newly added block gets connected to the previous block using its hash value. If the connection between blocks is changed, the blockchain verification fails. This helped us understand how hashing is used to maintain the integrity of data in a blockchain.